Copo: Joint Cost and Performance Optimization for Task Placement in Geo-Distributed Clouds

Bichen Wang[1] Jingzhou Wang[2,3,✉] Yu-E Sun[1,✉] He Huang[2,3]
[1]School of Rail Transportation, Soochow University, China
[2]School of Computer Science and Technology, Soochow University, China
[3]Key Laboratory of Data Intelligence and Advanced Computing in Provincial Universities, Soochow University, China
[✉]Correspondence to: jzwang6972@suda.edu.cn and sunye12@suda.edu.cn

2025 IEEE 33rd International Conference on Network Protocols (ICNP) | 979-8-3315-0376-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICNP65844.2025.11192448

***Abstract*—To provide a wide range of services for global users, cloud providers tend to build geo-distributed regions all over the world. With the rapid growth of cloud services, massive workloads and inter-region traffic have been introduced to current cloud networks, resulting in huge expenditure. Therefore, it is essential for a cloud provider to carefully place tasks and transfer traffic among regions to minimize the total operating costs. Existing solutions typically focus on optimizing either placement costs (*e.g.*, computing resources, and electricity) or bandwidth costs, and overlook performance metrics, which leads to increased overall operating costs or lower user QoS. To bridge the gap, this paper proposes Copo, a joint cost and performance optimization framework for tenant task placement in geo-distributed clouds. We first formalize the cost optimization problem as an undetermined multi-commodity flow problem which has never been studied before, and propose a graph transformation algorithm to reduce the complexity. Then we combine the cost optimization with the performance optimization as the final framework. The key idea of Copo is leveraging KKT conditions to transfer the bi-level optimization to a single level. To efficiently acquire the joint task placement and traffic transfer decisions, we leverage McCormick Envelope-based relaxation to design a randomized rounding-based approximation algorithm. Extensive experiments based on real-world data show the superior cost-efficiency and performance of Copo compared with state-of-the-art solutions.**

***Index Terms*—Geo-Distributed Cloud, Traffic Engineering, Task Placement, Cost-Efficiency, Performance**

I. Introduction

With the rapid development of cloud computing, large-scale cloud providers like Google Cloud Platform (GCP), Microsoft Azure, and Amazon Web Services (AWS) have made huge investments in building geographically distributed datacenters (geo-distributed DCs) to deliver a wide range of services, *e.g.*, searching, streaming, and online games to users throughout the world [1]. To achieve efficient resource management, service scalability, and also to meet specific data residency and compliance requirements, cloud providers propose the architecture *regions* [2], [3], *i.e.*, different physical locations around the world where DCs are clustered. For instance, Microsoft Azure has built regions in South/North America, Europe, Asia, and Oceania *etc.*, over 60 countries. With the advancement of networking and high-performance computing, an increasing number of enterprises/individuals are placing their services and tasks in regions on different continents to attract more global users [4].

Though providing global services can significantly increase revenue for cloud providers [5], this operation does not come without costs. In general, placing tasks in regions will consume a certain amount of resources, leading to corresponding costs. It is worth noting that the ways to measure consumed resources vary in different researches. For instance, the work [6] mainly focuses on the fine-grained resources (*e.g.*, CPU cores, memory, and disk), while the work [7] pays attention to relatively coarse-grained resources (*e.g.*, the number of virtual machines/containers). In addition, Luo *et al.* calculate energy-related metrics, *i.e.*, the electricity consumed by tasks [8]. For ease of description and without ambiguity, this paper uniformly calls the cost generated by placing tasks in regions the ***placement cost***. According to [8], the placement cost, especially the electricity cost generated by the intensive workloads, has been rapidly rising in the past decades, accounting for 70% of overall costs.

Moreover, due to the requirements of communication among tasks, the ***bandwidth cost*** is another key component of the total expenditure. Practically, the traffic between regions is essential for worldwide deployed applications and services [9]. Online applications such as social networks [10] keep generating a large volume of inter-datacenter traffic. The survey [11] highlights that nearly 70% of cloud providers have massive inter-region traffic ranging from 330 TB to 3.3 PB per month. To transfer data across regions, cloud providers need to purchase bandwidth from the Internet Service Provider (ISP) for the links they use [12]. leading to heavy cost for cloud providers [13]. According to Azure [14], the average bandwidth cost for inter-region traffic is about $0.2 per GB. For some large video streaming providers like YouTube and Netflix, it is estimated that the bandwidth cost for a year is over several million dollars [15].

Considering both the placement cost and the bandwidth cost, it is necessary for cloud providers to carefully place tenants' tasks and transfer traffic in a cost-efficient manner. Though there have been many cost-efficiency works for task placement in geo-distributed clouds, they all have the following shortcomings. **First, they only focus on one kind of cost, overlooking the other kind.** For instance,

979-8-3315-0376-5/25/$31.00 ©2025 IEEE

TanGo [8] formulates the electricity cost minimization for the task placement problem as a constrained mixed-integer non-linear programming problem while meeting various tenant requirements. However, TanGo does not take complex traffic routing into consideration, which may lead to non-optimal total costs. In comparison, COIN [16] tries to minimize the bandwidth cost for inter-region traffic transfer, but the insight of COIN is still limited since it cannot decide where to place tasks to further reduce placement cost. **Second, these works all overlook improving performance.** Even though works like TanGo and COIN consider satisfying multiple constraints like the bandwidth capacity constraint, traffic delay constraint, *etc.*, they cannot further improve performance metrics like load balancing and flow completion time. Consequently, the overall performance of clouds is relatively mediocre, which may not be able to provide enough availability or fault tolerance to handle events like traffic bursts [17] or crash failures [18]. Therefore, we aim to answer a meaningful yet unsolved question: Is it possible to determine how to jointly **place tenant tasks** and **transfer traffic** so that overall costs can be reduced significantly while **improving performance**?

To conquer these challenges, this paper proposes Copo, a task placement framework for reducing both placement and bandwidth costs. Furthermore, Copo can help improve certain kinds of performance metrics according to requirements. With the help of Copo, tenants can specify their various demands for tasks, and cloud providers can place tenant tasks and transfer inter-region traffic in a cost-effective manner while improving performance. The core design of Copo includes a general task placement framework and a near-optimal algorithm that could minimize the total costs while satisfying all the demands and constraints. In fact, the formulated problem of Copo is theoretically and mathematically complex, which has never been studied before, *i.e.*, the undetermined multi-commodity flow problem. In summary, we make the following contributions:

- We comprehensively summarize existing solutions on task placement in clouds, and show the advantages of Copo with a motivating example.
- We first formalize the cost optimization problem as an undetermined multi-commodity flow problem which has never been studied before, and propose a graph transformation algorithm to reduce the complexity.
- We then combine the cost optimization with performance optimization as a bi-level optimization formulation. Based on KKT conditions, we transfer the bi-level optimization to a single-level optimization problem.
- Based on McCormick Envelope-based relaxation and randomized rounding, we propose an efficient approximation algorithm to acquire the joint decisions on task placement and traffic transfer.
- Extensive experiments based on real-world data show the superior cost-efficiency and performance of Copo compared with state-of-the-art solutions.

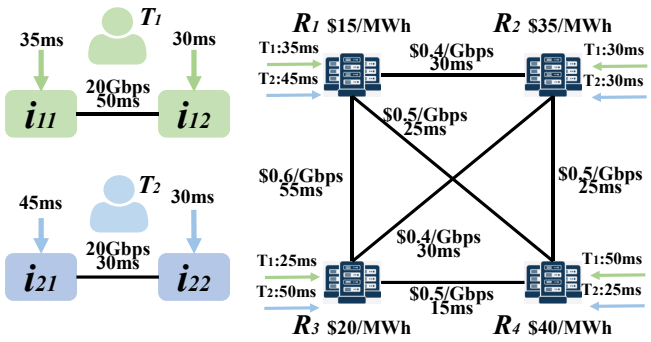


Fig. 1: A motivating example. *Left*: the access delay demand for each task and the bandwidth/delay demand between tasks. *Right*: the electricity cost of each region, the bandwidth cost among regions, the delay between tenants and regions, and the delay among regions.

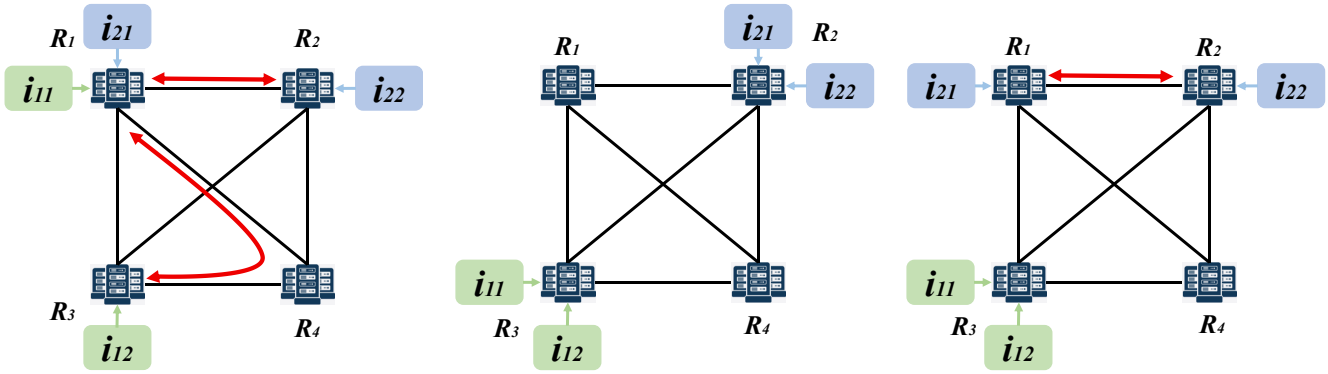
II. Background and Motivation

This section first discusses existing works on task placement in a geo-distributed cloud and then shows their limitations through a motivating example. Finally, we give an overview and workflow of Copo.

A. Current Solutions and Limitations

Overview of solutions comparison. In general, placement cost-aware solutions [6]–[8], [19]–[21] can efficiently decide how to place tasks in regions to save resource costs, but this kind of solution may lead to non-optimal total costs since the bandwidth cost is not taken into consideration. Though bandwidth cost-aware solutions [16], [22]–[25] can acquire traffic engineering solutions to reduce the bandwidth cost, they are not able to choose where to place tasks. Finally, performance-aware solutions [26]–[33] usually optimize one specific kind of performance metric, which cannot be customized according to different requirements. Also, this kind of solution is completely cost-oblivious, which may lead to higher expenditure. For comparison, our solution Copo enables a general task framework, taking both placement and bandwidth cost into consideration. Moreover, Copo can adjust the performance objective according to requirements, effectively solving all the shortcomings of related works, which will be demonstrated in detail as follows.

Placement Cost-Aware Solutions. The placement cost is a generalized concept, which can refer to fine-grained resources (*e.g.*, CPU, memory, and disk [6]), coarse-grained resources (*e.g.*, number of virtual machines/containers [7]) or electricity cost [8]. Some other studies consider adopting eco-friendly energies [19] to reduce the overall costs. The general approach of these works is to formulate the cost optimization problem as a combinatorial optimization problem with multiple constraints (*e.g.* resource capacity and task placement constraints), and the objective is to minimize corresponding costs. Then, they propose approximation or heuristic machine learning-based algorithms to acquire a cost-efficient solution. For instance, the authors of [8] formulate the task placement problem as a multiple knapsack problem and propose a



(a) TanGo (b) COIN (c) Copo

Fig. 2: Task placement and traffic transfer results of three solutions.

TABLE I: Comparison results of the motivating example.

Solutions	Placement				Communication		Costs		
	i_{11}	i_{12}	i_{21}	i_{22}	$i_{11} \leftrightarrow i_{12}$	$i_{21} \leftrightarrow i_{22}$	Electricity	Bandwidth	Total
TanGo [8]	R_1	R_3	R_1	R_2	(R_1, R_4, R_3)	(R_1, R_2)	\$85	\$28	\$113
COIN [16]	R_3	R_3	R_2	R_2	\	\	\$110	\$0	\$110
Copo (This paper)	R_3	R_3	R_1	R_2	\	(R_1, R_2)	\$90	\$8	\$98

submodular function-based greedy algorithm to solve it. Though these works can help reduce the placement cost and improve resource utilization, they have all overlooked the bandwidth cost. For instance, the work [6] only focuses on how to leverage a reinforcement learning-based approach for multidimensional resource allocation within a datacenter, overlooking the traffic demand among multiple datacenters.

Bandwidth Cost-Aware Solutions. Since online applications have generated enormous traffic between regions, how to optimize the bandwidth cost for transferring traffic has become another critical concern for cloud providers. With the wide adoption of the 95th-percentile billing method, how to optimally allocate traffic over links for cost saving has become more theoretically complex since the optimization problem has proven to be NP-Hard. The work [23] proposes CASCARA, which tries to minimize the bandwidth cost by directly solving the formulated integer programming (IP) problem. Based on CASCARA, Zhao *et al.* propose COIN [16] , the first traffic engineering framework integrated with multiple pricing schemes. The authors formulate the cost optimization problem as a mixed integer and linear programming (MILP) problem and design a partition rounding-based algorithm to solve it. However, these works can only optimize the bandwidth cost for traffic. They are not able to decide where to place these tasks.

Performance-Aware Solutions. According to [26], motivated by the rapidly growing trends of cloud services, the networking communities have made great efforts in the design of new task placement or traffic engineering solutions for geo-distributed clouds, especially to deliver enormous traffic efficiently. To this end, their solutions pursue research goals of improving resource utilization [27]–[29], meeting transfer deadlines [30]–[32], and minimizing transfer completion times [33]. Though these solutions can effectively optimize the goals above, they all lack flexibility since they only focus on one or several kinds of performance metrics. We cannot customize these objectives for different requirements. For instance, for a delay-sensitive application [34] like an online conference, the objective is to minimize the overall latency. For a bandwidth-sensitive application [35] like high-resolution video streaming, we need to guarantee load balancing and high utilization. Therefore, proposing a task placement framework with customizable objectives is urgently needed.

B. A Cost-Efficiency Motivating Example

We use a motivating example to show the cost-efficiency of Copo. In this example, we mainly consider the electricity cost as the placement cost. As shown in the left part of Fig. 1, there are two tenants T_1 and T_2. Each tenant has two tasks that need to be placed. We show the delay demand of each task and the traffic demand between the two tasks required by tenants. For instance, tenant T_1 has two tasks labeled by i_{11} and i_{12}. The access delays of task i_{11} and i_{12} are 35 ms and 30 ms, respectively. They need to occupy 20 Gbps bandwidth, and the communication delay demand is 50 ms. Furthermore, we assume that the electricity consumption of each task is 1 MWh for simplicity. As shown in the right part of Fig. 1, there are four regions denoted as R_1, R_2, R_3, R_4, the electricity prices of which are 15, 35, 20, and 40 (\$/MWh), respectively. We further show the bandwidth cost and the

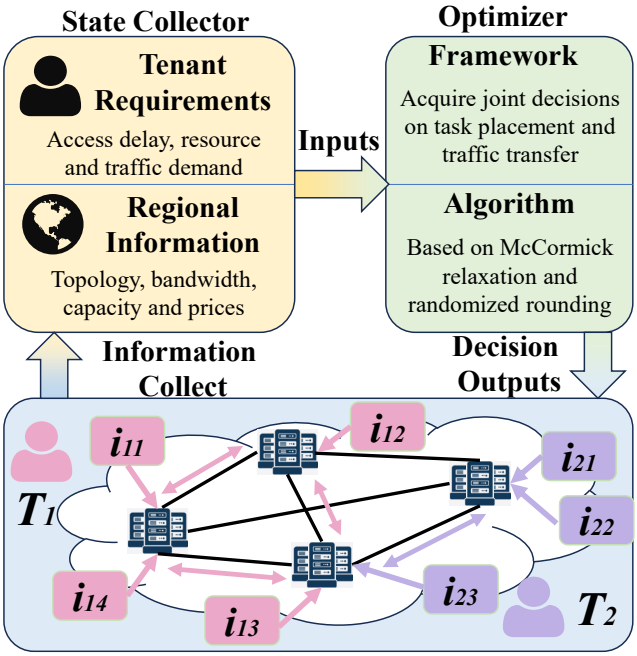


Fig. 3: Overview and workflow of Copo, including a state collector and a optimizer.

delay among regions, as well as the access delay between regions and tenants. For instance, the bandwidth cost between R_1 and R_2 is \$0.4/Gbps, and the delay is 30 ms. For tenant T_1 and T_2, the access delays to R_1 are 35 ms and 45 ms, respectively. For simplicity, we assume that the resources of regions and the bandwidth of links are both large enough. We compare Copo with state-of-the-art solutions, and the results are shown in Table I.

As a representative electricity cost-aware task placement solution, TanGo greedily chooses the available region with the lowest price to place tasks. The results are shown in Fig. 2(a). In this example, for task i_{11}, the available regions are R_1, R_2, R_3, among which R_1 has the lowest electricity price. Therefore, i_{11} will be placed in R_1. Similarly, i_{12}, i_{21}, i_{22} will be placed in R_3, R_1, R_2, respectively. After task placement, the traffic between i_{11} and i_{12} needs to be sent along $R_1 \leftrightarrow R_4 \leftrightarrow R_3$ to meet the traffic delay constraint. Since the electricity cost of $i_{11}, i_{12}, i_{21}, i_{22}$ is $\$15 + \$20 + \$15 + \$35 = \$85$, and the bandwidth costs of $R_1 \leftrightarrow R_4 \leftrightarrow R_3$ and $R_1 \leftrightarrow R_2$ are $\$10 + \$10 + \$8 = \$28$, the total costs are \$113.

Then, we consider how to apply COIN to this example. Since COIN tries to minimize bandwidth cost, it will try to place tasks with communication demand in the same region. With high-connectivity, low-latency networking within a region, the bandwidth cost and delay between two tasks in the same region can be ignored. Therefore, COIN will place i_{11} and i_{12} in R_3, and i_{21} and i_{22} will be placed in R_3. The total costs only involve electricity cost, which is $\$25 + \$25 + \$30 + \$30 = \$110$.

However, if we adopt Copo to jointly consider the electricity cost and the bandwidth cost, we can acquire a more cost-efficient solution. With Copo, task i_{11} and i_{12} will be placed in R_3, while i_{21}/i_{22} will be placed in R_1/R_2, respectively. The electricity cost of $i_{11}, i_{12}, i_{21}, i_{22}$ is $\$20 + \$20 + \$15 + \$35 = \$90$, and the bandwidth cost of $R_1 \leftrightarrow R_2$ is \$8. The total costs are \$98. Compared with TanGo and COIN, Copo reduces the total costs by 13.27% and 10.91%, respectively. This example fully demonstrates the potential of Copo to further reduce costs compared with existing works.

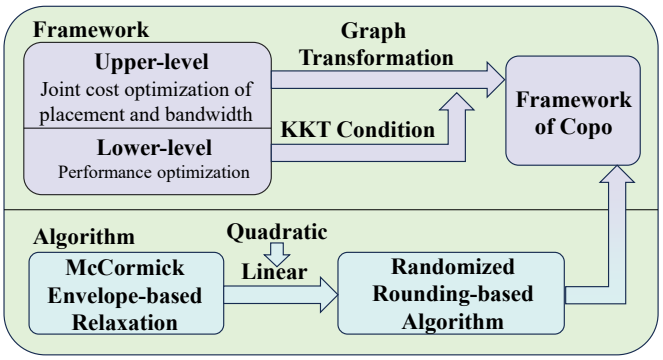


Fig. 4: Design of optimizer.

C. Overview and Workflow of Copo

We show the overview of Copo in Fig. 3. In general, Copo mainly includes two components: state collector and optimizer. The state collector is responsible for collecting tenant requirements (*e.g.*, the number of tasks, resource consumption, and access delay), and regional state information (*e.g.*, regional topology, bandwidth, and electricity price). The cloud providers could utilize the detailed information as inputs of the optimizer to make joint optimal placement and traffic transfer decisions. The carefully designed optimizer includes a framework and corresponding efficient algorithms.

Then, we briefly introduce the workflow of Copo. To make optimal decisions, the state collector first collects the regional information, including the regional topology, current resource prices (*e.g.*, hardware, electricity and bandwidth) and the delay/bandwidth between any pair of regions. Moreover, considering some information may vary over time, the state collector works on a periodical basis. For example, in regions with wholesale power markets, the state collector updates electricity prices hourly or every 15 minutes [36]. Once Copo obtains tenant inputs, including the specific information of tasks and corresponding demand (*e.g.*, resource consumptions or performance requirements), the optimizer can customize the framework to acquire the final solution with efficient algorithms. Based on the output, Copo is able to place tasks and determine how to transfer the traffic among regions.

The carefully designed optimizer is the core of Copo, also including two main components: the framework (Section III) and the algorithm (Section IV), as shown in Fig. 4. Specifically, the framework includes two levels of optimization problems. The upper-level tries to optimize the total costs, while the lower-level focuses on performance. We develop a graph transformation algorithm to modify the

upper-level optimization problem, then combine the lower-level optimization problem based on Karush–Kuhn–Tucker (KKT) conditions to formalize the framework of Copo, which is a quadratic programming problem. To effectively acquire the solution based on the framework, we first leverage McCormick Envelope-based relaxation to transfer the quadratic programming problem to a linear programming problem. Then, we propose the randomized rounding-based algorithm to acquire the final solution.

III. Framework Design

This section first defines the problem formulation of the upper-level optimization, *i.e.*, joint cost optimization of both placement and bandwidth (JCO-PB), and then analyzes its difficulty. Finally, we will show how to extend JCO-PB to the framework of Copo by combining the lower-level optimization, *i.e.*, the performance optimization.

A. Network Model

As shown in Fig. 3, the state collector collects necessary regional information and tenant requirements as the inputs of the proposed framework, which constitute the network model. The specific model parameters are listed as follows.

Regional Information. A typical geo-distributed cloud can be modeled by a graph $G = (V, E)$. Here V is the set of nodes, *i.e.*, the set of regions $R = \{r_1, r_2, ...\}$. The edge set E represents the link set between regions. In this paper, we mainly focus on the CPU cores as the resource constraint. Let C_r denote the number of CPU cores of region r. Note that C_r can be extended to a resource vector to represent different types of resources of region r, such as memory and disk. Furthermore, we take the electricity cost as the placement cost, and the unit electricity price of region r is denoted as p_r. For a link (u, w), the unit bandwidth cost is denoted as $p_{u,w}$. The corresponding delay and bandwidth capacity are denoted as $\tau_{u,w}$ and $b_{u,w}$, respectively. Without ambiguity, here u and w refer to regions.

Tenant Requirements. The set of tenants is denoted as $T = \{t_1, t_2, ...\}$. Since tenants need to deploy different tasks in clouds, we use $I = \{i_1, i_2, ...\}$ to denote the set of tasks and I^t to denote the set of tasks of tenant t. The tasks we considered in this paper include kinds of cloud services, like online games and video streaming, as well as LLM training and big data. The resource required by task i of tenant t is denoted by c_i^t, which can also be extended to a vector. The electricity consumption of i is q_i^t. For a region r, the access delay between tenant t and r is denoted as τ_r^t. The access delay demand between a tenant t and a task i is denoted as τ_i^t. Since two tasks may need to communicate with each other to transfer a certain amount of traffic, we use $J = \{j_1, j_2, ...\}$ to denote the communication demand set. For instance, if task i needs to communicate with i', then we have $j = (i, i')$. In addition, s_j and d_j represent the source and the sink of j, respectively. b_j represents the bandwidth demand of j, and τ_j denotes the delay demand.

Variables. Since the key step of JCO-PB is to determine which region to place tasks and how tasks communicate with each other, there exist *two sets of variables*. First, we use the variable $x_{i,r}^t \in \{0, 1\}$ to denote whether the task i of tenant t is deployed in region r. Then, the variable $y_{u,w}^j \in \{0, 1\}$ denotes whether the transmission path of j is through the edge (u, w). For simplicity, we consider the unsplittable model for flows [37].

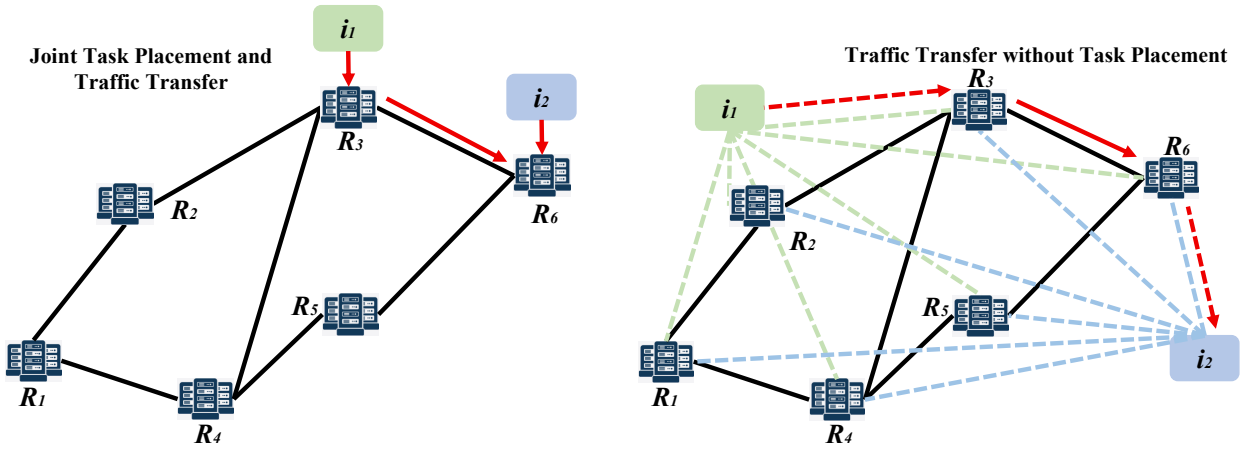
For clearer descriptions, we summarize the notations used in this paper in Appendix A. Then, we take Fig. 1 as an example to explain the notations. The node set is $V = \{R_1, R_2, R_3, R_4\}$, and the edge set is $E = \{(R_1, R_2), (R_1, R_3), (R_1, R_4), (R_2, R_3), (R_2, R_4), (R_3, R_4)\}$. The unit electricity price of R_1 is $p_1 =$ \$15/MWh. The unit bandwidth price of edge (R_1, R_2) is $p_{1,2} =$ \$0.4/Gbps, and the delay is $\tau_{1,2} = 30$ ms. Tenant T_1 has two tasks $\{i_{11}, i_{12}\}$ that need to be placed. The access delay demands of the two tasks are $\tau_{11}^1 = 35$ ms and $\tau_{12}^1 = 30$ ms. The delay between tenant T_1 and region R_1 is $\tau_1^1 = 35$ ms. Since the two tasks need to communicate with each other, the communication demand is denoted as $j = (i_{11}, i_{12})$. The bandwidth and delay demand are $b_j = 20$ Gbps and $\tau_j = 50$ ms.

B. Upper-Level Optimization

Now we introduce how to design the upper-level optimization with the inputs of regional information and tenant requirements, *i.e.*, the JCO-PB problem. We decouple the original problem mainly because the solution lies on the boundary of the Pareto frontier of a multi-objective optimization problem, corresponding to the point with the lowest possible cost. In this paper, we mainly regard the electricity cost as the placement cost.

Constraints. We first consider the following constraints in the upper-level optimization:

- *Task Placement Constraint:* The task i should be placed in one region.
$$\sum_r x_{i,r}^t = 1, \forall i \in I^t, t \tag{1}$$
- *Region Computing Constraint:* The placed tasks will consume a certain number of CPU cores. It follows:
$$\sum_t \sum_i x_{i,r}^t c_i^t \le C_r, \forall r \tag{2}$$
- *Flow Conservation Constraints:* For any communication demand j in clouds, flow conservation constraints must be satisfied as follows.
$$\begin{cases} \sum_w y_{u,w}^j - \sum_w y_{w,u}^j = 1, \forall j, u = s_j \\ \sum_w y_{u,w}^j - \sum_w y_{w,u}^j = 0, \forall j, u \ne s_j, d_j \\ \sum_w y_{w,u}^j - \sum_w y_{u,w}^j = 1, \forall j, u = d_j \end{cases} \tag{3}$$
- *Access Delay Constraint:* The tenants may require different access delays to their tasks according to the specific service demand. Therefore, the tasks should be placed in regions within a certain range. It follows:
$$x_{i,r}^t \tau_r^t \le \tau_i^t, \forall i \in I^t, t, r \tag{4}$$
- *Traffic Delay Constraint:* The communication delay between two tasks from the same tenant should not exceed the delay posed by the tenant. It follows:
$$\sum_{(u,w)\in E} y_{u,w}^j \tau_{u,w} \le \tau_j, \forall j \tag{5}$$



(a) Original graph with task placement and traffic transfer

(b) Augmented graph with traffic transfer without task placement

Fig. 5: An example of graph transformation. In the original graph, we have 6 nodes and 7 edges, and in the augmented graph, we have 8 nodes (including 2 virtual nodes) and 19 edges (including 12 virtual edges).

- *Traffic Bandwidth Constraint:* The traffic over any links (u, w) should not exceed the bandwidth capacity $b_{u,w}$, *i.e.*,

$$\sum_{j} y_{u,w}^{j} b_j \leq b_{u,w}, \forall (u,w) \in E \tag{6}$$

Costs. In this paper, we mainly focus on optimizing both electricity and bandwidth costs generated by placement and traffic transfer.

- *electricity cost:* We calculate the total electricity cost of all tasks as follows.

$$E_1 = \sum_{r} \sum_{t} \sum_{i \in I^t} x_{i,r}^{t} q_i^t p_r \tag{7}$$

- *bandwidth cost:* To transfer traffic between two regions, we need to pay corresponding bandwidth cost.

$$E_2 = \sum_{j} \sum_{(u,w) \in E} y_{u,w}^{j} b_j p_{u,w} \tag{8}$$

Combined with the constraints and objective, we can formulate the JCO-PB problem as follows:

$$\min E_1 + E_2$$
$$S.t. \begin{cases} Eqs.\ (1), (2), (3), (4), (5), (6), (7), (8) & \\ x_{i,r}^{t} \in \{0,1\}, & \forall i, t, r \\ y_{u,w}^{j} \in \{0,1\}, & \forall j, (u,w) \in E \end{cases} \tag{9}$$

It is worth noting that the flow conservation constraints are actually not *determined*. Since we need to jointly determine the placement of tasks and how to transfer traffic, we do not know the source and the sink of a communication demand j in the first place, *i.e.*, s_j and d_j are actually variables.

Theorem 1: The JCO-PB problem is NP-Hard

Due to limited space, we omit the detailed proof here. In fact, we must note that this is an extremely complex problem. Even if we ignore the bandwidth cost and the corresponding bandwidth constraints, the problem remains NP-Hard, since the multiple knapsack problem is a special case of simplified JCO-PB [8]. And if we ignore the electricity cost and have already acquired the task placement solution, the problem becomes the classical Multi-Commodity Flow (MCF) problem, which is also NP-Hard. To our best knowledge, this is a new problem that has never been studied before. Compared with the classical MCF problem, our problem does not know the sources and the sinks of all commodities in the first place. Thus, we call it the *undetermined multi-commodity flow (UMCF)* problem.

Algorithm 1 Graph Transformation Algorithm

Input: The graph of network model $G = (V, E)$ and task set I

```
Create a copy of G, denoted as G' = (V', E')
for Each i ∈ I do
    Create a virtual node i
    V' = V' ∪ i
    for Each r ∈ R do
        Connect i and R with a virtual edge (i, R)
        E' = E' ∪ (i, R)
Return the augmented graph G'
```

C. Graph Transformation

To deal with the UMCF-based JCO-PB problem, we show a graph theory-based transformation algorithm by introducing the *virtual nodes and edges*. The algorithm is formally shown in Alg. 1. Recall that the modeled graph G only consists of regions as nodes and links as edges. In our augmented graph $G' = \{V', E'\}$, we let all tasks be virtual nodes, *i.e.*, $V' = R \cup I$. For each virtual node, it will connect to every node in R with a virtual edge. For instance, in Fig. 5, we have two tasks denoted by i_1 and i_2. The graph contains six nodes $R_1, R_2, ..., R_6$ and seven edges. If we try to directly solve the JCO-PB problem, we need to jointly determine how to place tasks and transfer traffic, as shown in Fig. 5(a). However, if we let i_1 and i_2 become two virtual nodes and connect them with all other nodes of R, the original problem is transformed into an equivalent form, where the commodity flows have

specific sources and sinks, *i.e.*, the corresponding task nodes. As shown in Fig. 5(b), if the path between nodes (i, i') is (i_1, R_3, R_6, i_3), which includes two virtual edges (i_1, R_3) and (R_6, i_2), it actually means that i_1/i_2 is placed in R_3/R_6, respectively. Obviously, the transformation can substantially reduce the complexity of the problem.

With the augmented graph, we can re-write the flow conservation constraints as follows:

$$\begin{cases} \sum_{w\in R} y^j_{u,w} - \sum_{w\in R} y^j_{w,u} = 1, & \forall j.u = i \\ \sum_{w\in R} y^j_{u,w} - \sum_{w\in R} y^j_{w,u} = 0, & \forall j, u \neq i, i' \\ \sum_{w\in R} y^j_{w,u} - \sum_{w\in R} y^j_{u,w} = 1, & \forall j, u = i' \end{cases} \tag{10}$$

Moreover, we can determine the value of $x^t_{i,r}$ based on $y^j_{u,w}$. Specifically, if $y^j_{i,r} = 1$ or $y^j_{r,i} = 1$, it means $j = (i, i')$ goes through the virtual edge (i, r) or (r, i), *i.e.*, task i is placed in r. It follows:

$$\begin{cases} y^j_{i,r} \leq x^t_{i,r}, \forall j = (i, i'), r \in R \\ y^j_{r,i} \leq x^t_{i,r}, \forall j = (i, i'), r \in R \end{cases} \tag{11}$$

We improve our original formulation defined in Eq. (9) as follows:

$$\min E_1 + E_2$$

$$S.t. \begin{cases} Eqs.\ (1), (2), (4), (5), (6), (7), (8) & \\ Eqs.\ (10), (11) & \\ x^t_{i,r} \in \{0, 1\}, & \forall i \in I^t, t, r \\ y^j_{u,w} \in \{0, 1\}, & \forall j, (u, w) \in E' \end{cases} \tag{12}$$

Compared with Eq. (9), Eq. (12) can specify the sources and sinks of all communication demands, which can be reduced to the MCF problem to prove its NP-Hardness.

D. Lower-Level Optimization and KKT Conditions

According to [26], there exist multiple metrics when transferring a large volume of traffic among geo-distributed clouds, such as utilization and flow completion time. In this paper, we take link load balancing as an example to show how Copo works, which can be customized according to requirements. We first write the lower-level optimization problem:

$$\min \zeta$$

$$\begin{cases} \sum_j y^j_{u,w} b_j \leq \zeta b_{u,w}, (u, w) \in E \\ \zeta \in [0, 1] \end{cases} \tag{13}$$

Leveraging the Karush–Kuhn–Tucker (KKT) conditions [38], we can transform the bi-level optimization problem into a single-level optimization problem. The use of KKT conditions enables the lower-level optimization to be equivalently transformed into a set of constraints in the upper-level problem. This allows the bi-level model to be converted into a single-level formulation, facilitating efficient and unified solving while preserving the trade-off dynamics between objectives. According to the KKT conditions, the Lagrangian function can be written as:

$$L\left[\zeta, y^j_{u,w}, \lambda_{u,w}, \alpha, \beta\right] = \zeta + \sum_{(u,w)\in E} \lambda_{u,w} \left[\sum_j y^j_{u,w} b_j - \zeta b_{u,w}\right] + \alpha(\zeta - 1) - \beta\zeta \tag{14}$$

We can further acquire the KKT equations and transfer to:

$$\begin{cases} \sum_{(u,w)\in E} \lambda_{u,w} b_{u,w} = 1 \\ 0 \leq \sum_{(u,w)\in E} \lambda_{u,w} (\sum_j y^j_{u,w} b_j) \leq 1 \\ \lambda_{u,w} \geq 0 \end{cases} \tag{15}$$

Due to the limited space, we omit the related proofs of the feasibility of applying KKT conditions. Readers can refer to [38] for the detailed proof.

E. Framework of Copo

Now we introduce the complete framework of Copo. Combining Eq. (12) and Eq. (15), the complete framework of Copo is as follows.

$$\min E_1 + E_2$$

$$S.t. \begin{cases} \sum_r x^t_{i,r} = 1, & \forall i \in I^t, t \\ \sum_t \sum_i x^t_{i,r} c^t_i \leq C_r, & \forall r \\ \sum_{w\in R} y^j_{u,w} - \sum_{w\in R} y^j_{w,u} = 1, & \forall j, u = i \\ \sum_{w\in R} y^j_{u,w} - \sum_{w\in R} y^j_{w,u} = 0, & \forall j, u \neq i, i' \\ \sum_{w\in R} y^j_{w,u} - \sum_{w\in R} y^j_{u,w} = 1, & \forall j, u = i' \\ y^j_{i,r} \leq x^t_{i,r}, & \forall j, r \\ y^j_{r,i} \leq x^t_{i,r}, & \forall j, r \\ x^t_{i,r} \tau^t_r \leq \tau^t_i, & \forall i \in I^t, t, r \\ \sum_{(u,w)\in E} y^i_{u,w} \tau_{u,w} \leq \tau_j, & \forall j \\ \sum_j y^j_{u,w} b_j \leq b_{u,w}, & \forall (u, w) \in E \\ E_1 = \sum_r \sum_t \sum_{i\in T^t} x^t_{i,r} q^t_i p_r & \\ E_2 = \sum_j \sum_{(u,w)\in E} y^j_{u,w} b_j p_{u,w} & \\ x^t_{i,r} \in \{0, 1\}, & \forall i \in I^t, t, r \\ y^j_{u,w} \in \{0, 1\}, & \forall j, (u, w) \in E' \\ \sum_{(u,w)\in E} \lambda_{u,w} b_{u,w} = 1 & \\ 0 \leq \sum_{(u,w)\in E} \lambda_{u,w} (\sum_j y^j_{u,w} b_j) \leq 1 & \\ \lambda_{u,w} \geq 0 & \end{cases} \tag{16}$$

The problem defined in Eq. (16) is a very complex mixed quadratic integer programming (MQIP) problem. We will show how to design an efficient approximation algorithm to solve it in the next section. Copo models cost and performance as a bi-level optimization problem as a case study in this paper, yet other objectives, *e.g.*, latency, resource utilization, and makespan—can also be formulated as linear programs, making them compatible with Copo.

IV. Algorithm Design

This section shows how to design an efficient approximation algorithm based on McCormick Envelope-based relaxation and randomized rounding.

A. McCormick-based Relaxation

Since Eq. (16) is a quadratic programming problem, we need to leverage the McCormick Envelope-based re-

Algorithm 2 Randomized Rounding-based Algorithm

Input: The formulation Eq. (16) and Eq. (19)

1: **Step 1**: Solving the Relaxed Formulation
2: Replace $x_{i,r}^t \in \{0,1\}$ and $y_{u,w}^j \in \{0,1\}$ with $x_{i,r}^t \in [0,1]$ and $\{y_{u,w}^j \in [0,1]\}$
3: Solve the relaxed formulation and acquire the optimal solution $\{\widetilde{x_{i,r}^t}\}$ and $\widetilde{y_{u,w}^j}$
4: **Step 2**: Acquiring the feasible solution
5: **for** Each $x_{i,r}^t$ **do**
6: Round $\widetilde{x_{i,r}^t}$ to 1 with probability of $\widetilde{x_{i,r}^t}$
7: **for** Each $y_{u,w}^j$ **do**
8: Round $\widetilde{y_{u,w}^j}$ to 1 with probability of $\widetilde{y_{u,w}^j}$
9: **Return** $\widehat{x_{i,r}^t}$ and $\widehat{y_{u,w}^j}$

laxation [39] method to transform Eq. (16) to a linear programming problem. Consider the quadratic constraint:

$$0 \leq \sum_{(u,w)\in E} \lambda_{u,w}(\sum_j y_{u,w}^j b_j) \leq 1 \tag{17}$$

We can acquire the upper bound of $\lambda_{u,w}$ according to the constraint $\sum_{(u,w)\in E} \lambda_{u,w} b_{u,w} = 1$. Therefore, for any $(u,w) \in E$, $\lambda_{u,w} \in \left[0, \frac{1}{b_{u,w}}\right]$. Moreover, we have the lower and the upper bound of $\sum_j y_{u,w}^j b_j$ as follows:

$$0 \leq \sum_j y_{u,w}^j b_j \leq \sum_j b_j \tag{18}$$

Then, we can regard $\lambda_{u,w} \sum_j y_{u,w}^j b_j$ as a new variable, denoted as $\Lambda_{u,w}$. We have the following auxiliary constraints:

$$\begin{cases} 1 \geq \sum_{(u,w)\in E} \Lambda_{u,w} \geq 0 \\ 1 \geq \sum_{(u,w)\in E} \Lambda_{u,w} \geq \\ \sum_{(u,w)\in E} \left[\frac{1}{b_{u,w}} \sum_j y_{u,w}^j b_j + \lambda_{u,w} \sum_j b^j - \frac{\sum_j b_j}{b_{u,w}}\right] \\ 0 \leq \sum_{(u,w)\in E} \Lambda_{u,w} \leq \sum_{(u,w)\in E} \sum_j b^j \\ 0 \leq \sum_{(u,w)\in E} \Lambda_{u,w} \leq \frac{\sum_j y_{u,w}^j b_j}{b_{u,w}} \end{cases} \tag{19}$$

We replace the original constraint $0 \leq \sum_{(u,w)\in E} \lambda_{u,w}(\sum_j y_{u,w}^j b_j) \leq 1$ with $0 \leq \sum_{(u,w)\in E} \Lambda_{u,w} \leq 1$. Then, we add the four auxiliary constraints in Eq. (19). The original formulation Eq. (16) becomes a mixed-integer and linear programming problem.

B. Randomized Rounding-based Algorithm

We present an approximation algorithm based on randomized rounding for Copo. In specific, the first step is to relax variables by replacing integer constraints $x_{i,r}^t \in \{0,1\}$ and $y_{u,w}^j \in \{0,1\}$ with linear constraints $x_{i,r}^t \in [0,1]$ and $y_{u,w}^j \in [0,1]$. In this way, we can solve it with a linear program solver (*e.g.*, PuLP or Gurobi), acquiring the optimal but fractional solutions $\widetilde{x_{i,r}^t}$ and $\widetilde{y_{u,w}^j}$. In the second step, we obtain feasible solutions $\widehat{x_{i,r}^t}$ and $\widehat{y_{u,w}^j}$ by rounding the variables according to the optimal value. The formal algorithm is shown in Alg. 2.

C. Performance Analysis

Theorem 2: The joint optimization problem is solvable in polynomial time.

The worst-case time complexity of solving the LP is only $O(n^{3.5})$, where n is the number of variables. Additionally, the algorithm can be executed in a batch-processing manner. For instance, for every 5-minute workload, we execute the corresponding algorithm, thus avoiding large-scale inputs and longer execution time.

Theorem 3: The approximation ratio of Alg. 2 is $O(\log |E|)$, where $|E|$ is the number of edges of the augmented graph.

Due to the limited space, we omit the detailed proof here. Readers can refer to [40] for the performance analysis of randomized rounding.

V. Performance Evaluation

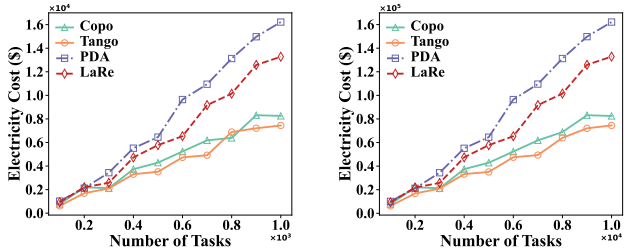
This section first introduces the simulation settings for performance comparison and then evaluates Copo by comparing it with existing methods through extensive simulations.

A. Simulation Settings

1) Datasets: In our simulation experiments, we mainly consider the following datasets for evaluation, *i.e.*, the small- and large-scale region dataset and the small- and large-scale task dataset, which are described in detail in Appendix B.

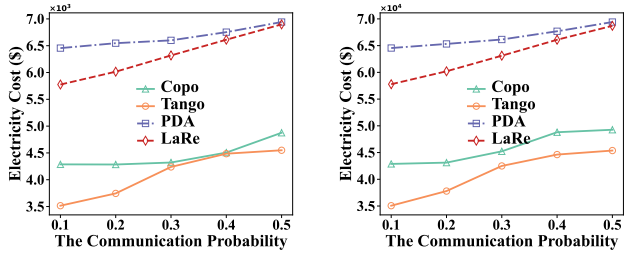
2) Performance Metrics: We mainly consider the following performance metrics in our simulations.

- **Electricity Cost**: We calculate the electricity cost as the placement cost incurred by different task deployment methods when tasks are successfully deployed within regions. This metric reflects the consumption of electrical resources during task placement. It constitutes an important part of the total costs and serves as a key indicator of the algorithm's optimization capability under various constraints. We expect that Copo will not perform significantly worse than the baseline methods that specifically optimize for electricity cost.
- **Bandwidth Cost**: This metric quantifies the bandwidth consumption resulting from communication between task pairs. We compare the bandwidth costs of various algorithms to evaluate their effectiveness in addressing communication demands and optimizing network resource utilization. Copo is expected to deliver performance comparable to algorithms that explicitly minimize bandwidth cost, while outperforming those that do not incorporate bandwidth cost as a primary optimization objective.
- **Total Costs**: The total costs are the weighted sum of the electricity cost and the bandwidth cost, and serves as the most intuitive and important metric for evaluating the overall performance of each algorithm. By comparing the total costs, we can directly assess the effectiveness of different algorithms in both task deployment and communication optimization. We hope



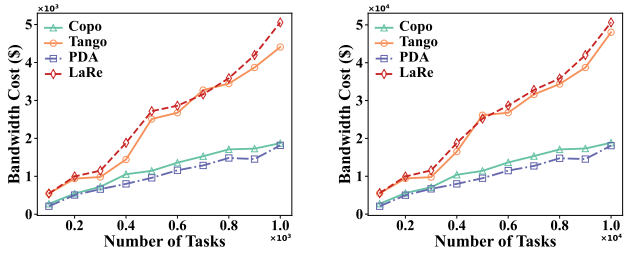
(a) The Small Scale (b) The Large Scale

Fig. 6: Electricity Cost vs. No. of Tasks for Diff. Scales



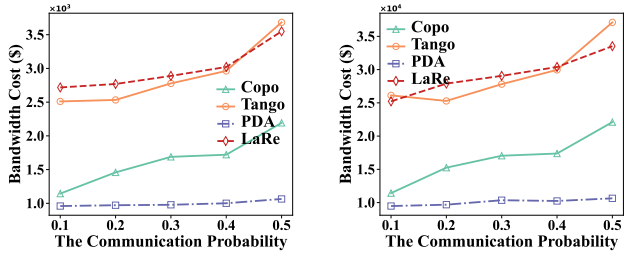
(a) The Small Scale (b) The Large Scale

Fig. 7: Electricity Cost vs. Comm. Pr. for Diff. Scales



(a) The Small Scale (b) The Large Scale

Fig. 8: Bandwidth Cost vs. No. of Tasks for Diff. Scales



(a) The Small Scale (b) The Large Scale

Fig. 9: Bandwidth Cost vs. Comm. Pr. for Diff. Scales

Copo can achieve the best performance in terms of total costs.

- **Load Balancing Ratio (LBR)**: Recall that we take load balancing as an example of performance metrics, which can be customized according to requiremets. This metric evaluates the ability of each algorithm to balance communication loads across network links, measured by how evenly the traffic is distributed. A lower load balancing ratio indicates better link utilization and reduced risk of congestion. We expect that Copo demonstrates stable and competitive performance in terms of load balancing among links.

3) Benchmarks: We compare Copo with the following representative baseline solutions:

- **TanGo** [5]: TanGo exploits spatial heterogeneity in regional electricity pricing by strategically allocating tasks to data center regions with lower energy costs. Concurrently, it ensures strict adherence to a range of tenant-specific constraints, including latency bounds, fault-tolerance requirements, and regional resource availability.
- **PDA** [22]: PDA is designed to minimize bandwidth cost by optimizing the scheduling of data transmissions, ensuring that all communication requirements are met within their respective deadlines while minimizing bandwidth utilization. The algorithm achieves this by selecting efficient transmission paths and scheduling strategies that reduce peak bandwidth usage, thereby lowering the overall cost of inter-task communication in bandwidth-constrained environments.
- **Lagrangian Relaxation(LaRe)** [41]: LaRe aims to achieve load balancing across inter-region communication links by minimizing the disparity in bandwidth utilization, while strictly satisfying communication requirements among interdependent tasks.

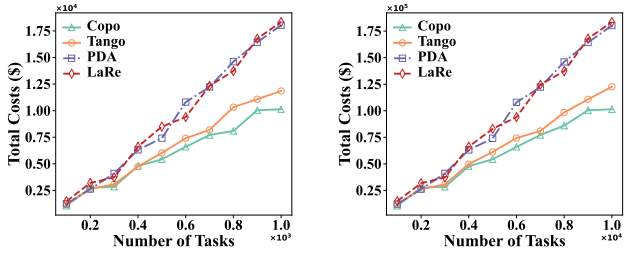
4) Evaluation Factors: We mainly evaluate the metrics above in terms of two factors, *i.e.*, number of tasks and communication probability. In our simulations, communication probability denotes the likelihood that an arbitrary pair of tasks has a communication requirement, reflecting the number of communication demands.

B. Performance Comparison

We mainly focus on the following evaluation metrics that are closely related to the experiments:

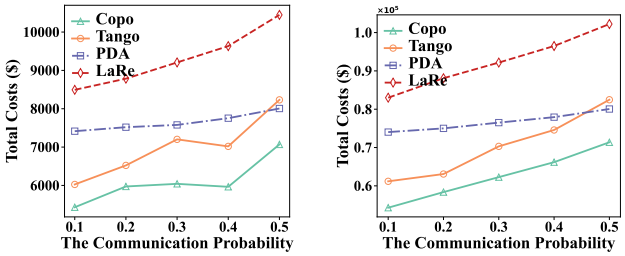
Electricity Cost versus Number of Tasks and Communication Probability (Across Different Scales). As depicted in Figs. 6–7, PDA and LaRe incur substantially higher electricity cost compared to TanGo and Copo. Copo achieves up to a 49% reduction in the electricity cost compared to PDA. This difference is primarily because PDA and LaRe do not incorporate the electricity cost as a clearly defined optimization objective during task placement. In contrast, although Copo simultaneously optimizes the electricity cost, the bandwidth cost, and the link load balance, its electricity cost remains comparable to that of TanGo, with a minimum difference of only 1.5%. This indicates that when task resource demands significantly exceed communication demands, Copo can still achieve near-optimal electricity cost performance within a multiobjective optimization framework.

Bandwidth Cost versus Number of Tasks and Communication Probability (Across Different Scales). As shown in Figs. 8–9, the bandwidth cost of PDA and Copo is significantly lower than those of TanGo and LaRe. Specifically, Copo achieves up to 60.74% lower bandwidth cost



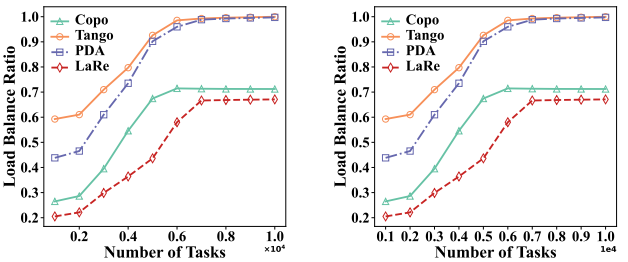
(a) The Small Scale (b) The Large Scale

Fig. 10: Total Costs vs. No. of Tasks for Diff. Scales



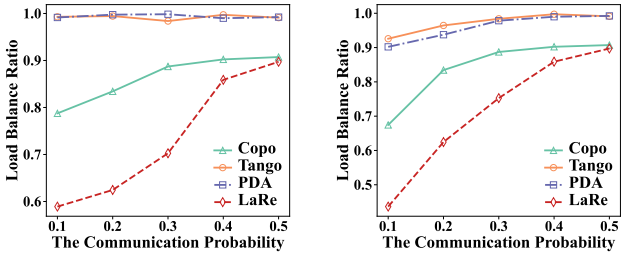
(a) The Small Scale (b) The Large Scale

Fig. 11: Total Costs vs. Comm. Pr. for Diff. Scales



(a) The Small Scale (b) The Large Scale

Fig. 12: LBR vs. No. of Tasks for Diff. Scales



(a) The Small Scale (b) The Large Scale

Fig. 13: LBR vs. Comm. Pr. for Diff. Scales

compared to TanGo and up to 62.78% lower compared to LaRe. The difference is observed because both PDA and Copo explicitly incorporate bandwidth cost as an optimization objective during task placement. Despite simultaneously optimizing for electricity cost and load balancing, Copo is still able to maintain a low bandwidth cost. The bandwidth cost of PDA is only about 10% lower than that of Copo. These results indicate that in scenarios where communication demand significantly exceeds bandwidth capacity, Copo can effectively minimize bandwidth cost within a multiobjective optimization framework.

Total Costs versus Number of Tasks and Communication Probability (Across Different Scales). As shown in Figs. 10–11, Copo consistently outperforms the three baseline algorithms in total cost. This is mainly due to the coordinated optimization of electricity and bandwidth cost. Specifically, as the task scale increases, Copo reduces the total costs by up to 14.50% compared to TanGo, 43.82% compared to PDA, and 44.74% compared to LaRe. This is because resource demand dominates the task data set, which favors TanGo over the other baselines. In addition, with increasing communication probability, the cost of the bandwidth becomes more important. In this case, Copo reduces the total cost by up to 11% compared to PDA.

Load Balancing Ratio versus Number of Tasks and Communication Probability (Across Different Scales). As illustrated in Figs. 12–13, Copo consistently achieves higher load balancing ratios than TanGo and PDA, although it is slightly inferior to LaRe. With the increase in task scale, TanGo and PDA tend to adopt deployment strategies primarily driven by cost minimization, resulting in load balancing ratios gradually approaching 100%. In contrast, Copo employs a comprehensive optimization strategy that balances both load balancing ratio and cost. As task volume and inter-task communication probability increase, Copo's load balancing ratio progressively converges with that of LaRe, with the difference between the two ultimately narrowing to approximately 4%. These results indicate that Copo can effectively maintain a balanced allocation of bandwidth resources within the system while simultaneously considering cost efficiency.

VI. Conclusion

This paper proposes Copo, a joint cost and performance optimization framework. We first formalize upper-level optimization, *i.e.*, the JCO-PB problem to minimize both placement and bandwidth costs. Due to its complexity, we develop the graph transformation algorithm to transfer the undetermined multi-commodity flow problem to the classical multi-commodity flow problem. Then, we combine JCO-PB with the lower-level optimization problem to further improve performance based on KKT conditions. Finally, we leverage McCormick Envelope-based relaxation and randomized rounding algorithm to acquire a near-optimal solution. Based on real-world datasets, we conduct extensive simulation experiments to show the superior cost-efficiency and performance compared with state-of-the-art solutions.

Acknowledgment

We sincerely thank the anonymous reviewers and shepherd who helped improve this paper. The corresponding authors of this paper are Jingzhou Wang and Yu-E Sun. This work is supported by the National Natural Science Foundation of China (NSFC) under Grant No. 62332013, and supported by Project Funded by the Priority Academic Program Development of Jiangsu Higher Education Institutions (PAPD).

REFERENCES

[1] J. Wang, G. Zhao, H. Xu, Y. Zhao, X. Yang, and H. Huang, "Trust: Real-time request updating with elastic resource provisioning in clouds," in *IEEE INFOCOM 2022*, pp. 620–629.

[2] J. Nazir, M. W. Iqbal, T. Alyas, M. Hamid, M. Saleem, S. Malik, and N. Tabassum, "Load balancing framework for cross-region tasks in cloud computing," *Computers, Materials & Continua*, vol. 70, no. 1, pp. 1479–1490, 2022.

[3] A. Newell, D. Skarlatos, J. Fan, P. Kumar, M. Khutornenko, M. Pundir, Y. Zhang, M. Zhang, Y. Liu, L. Le, B. Daugherty, A. Samudra, P. Baid, J. Kneeland, I. Kabiljo, D. Shchukin, A. Rodrigues, S. Michelson, B. Christensen, K. Veeraraghavan, and C. Tang, "Ras: Continuously optimized region-wide datacenter resource allocation," in *SOSP 2021*. New York, NY, USA: Association for Computing Machinery, 2021, p. 505–520.

[4] J. Wang, G. Zhao, H. Xu, and H. Wang, "Leaf: Improving qos for reconfigurable datacenters with multiple optical circuit switches," in *2024 IEEE/ACM IWQoS*. IEEE, 2024, pp. 1–10.

[5] S. Mithas, J. Whitaker, and A. Tafti, "Information technology, revenues, and profits: Exploring the role of foreign and domestic operations," *Information Systems Research*, vol. 28, no. 2, pp. 430–444, 2017. [Online]. Available: https://doi.org/10.1287/isre.2017.0689

[6] W. Wei, H. Gu, K. Wang, J. Li, X. Zhang, and N. Wang, "Multi-dimensional resource allocation in distributed data centers using deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1817–1829, 2022.

[7] Z. Hu, B. Li, and J. Luo, "Time-and cost-efficient task scheduling across geo-distributed data centers," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 3, pp. 705–718, 2017.

[8] L. Luo, G. Zhao, H. Xu, Z. Yu, and L. Xie, "Tango: A cost optimization framework for tenant task placement in geo-distributed clouds," in *IEEE INFOCOM 2023*, pp. 1–10.

[9] A. Greenberg, J. Hamilton, D. A. Maltz, and P. Patel, "The cost of a cloud: research problems in data center networks," *SIGCOMM Comput. Commun. Rev.*, vol. 39, no. 1, p. 68–73, Dec. 2009. [Online]. Available: https://doi.org/10.1145/1496091.1496103

[10] M. Noormohammadpour and C. S. Raghavendra, "Datacenter traffic control: Understanding techniques and tradeoffs," *IEEE Communications Surveys Tutorials*, vol. 20, no. 2, pp. 1492–1525, 2018.

[11] F. Research, "The future of data center wide-area networking," http://www.forrester.com, MAY 2025.

[12] W. Li, X. Zhou, K. Li, H. Qi, and D. Guo, "Trafficshaper: Shaping inter-datacenter traffic to reduce the transmission cost," *IEEE/ACM Transactions on Networking*, vol. 26, no. 3, pp. 1193–1206, 2018.

[13] W. Li, K. Li, D. Guo, G. Min, H. Qi, and J. Zhang, "Cost-minimizing bandwidth guarantee for inter-datacenter traffic," *IEEE Transactions on Cloud Computing*, vol. 7, no. 2, pp. 483–494, 2019.

[14] "Bandwidth costs," https://azure.microsoft.com/en-us/pricing/details/bandwidth/, MAY 2025.

[15] R. Miller, "Youtube bandwidth costs," https://www.datacenterknowledge.com/business/youtube-s-bandwidth-cheap-but-not-free, MAY 2025.

[16] G. Zhao, J. Wang, H. Xu, Z. Yu, and C. Qiao, "Coin: Cost-efficient traffic engineering with various pricing schemes in clouds," in *IEEE INFOCOM 2023*, pp. 1–10.

[17] D. Georgantas and P. Baziana, "Traffic burstiness study of an efficient bandwidth allocation mac scheme for wdm datacenter networks," in *2023 MeditCom*. IEEE, 2023, pp. 169–174.

[18] M. Kirti, A. K. Maurya, and R. S. Yadav, "Fault-tolerance approaches for distributed and cloud computing environments: A systematic review, taxonomy and future directions," *Concurrency and Computation: Practice and Experience*, vol. 36, no. 13, p. e8081, 2024.

[19] J. Bian, L. Wang, S. Ren, and J. Xu, "Cafe: Carbon-aware federated learning in geographically distributed data centers," in *Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems*, 2024, pp. 347–360.

[20] J. Gao, H. Wang, and H. Shen, "Smartly handling renewable energy instability in supporting a cloud datacenter," in *2020 IEEE international parallel and distributed processing symposium (IPDPS)*, pp. 769–778.

[21] J. Wang, G. Zhao, H. Xu, Y. Zhai, Q. Zhang, H. Huang, and Y. Yang, "A robust service mapping scheme for multi-tenant clouds," *IEEE/ACM Transactions on Networking*, vol. 30, no. 3, pp. 1146–1161, 2021.

[22] Z. Yang, Y. Cui, X. Wang, Y. Liu, M. Li, S. Xiao, and C. Li, "Cost-efficient scheduling of bulk transfers in inter-datacenter wans," *IEEE/ACM Transactions on Networking*, vol. 27, no. 5, pp. 1973–1986, 2019.

[23] R. Singh, S. Agarwal, M. Calder, and P. Bahl, "Cost-effective cloud edge traffic engineering with cascara," in *18th NSDI*, 2021, pp. 201–216.

[24] W. Li, X. Zhou, K. Li, H. Qi, and D. Guo, "Trafficshaper: Shaping inter-datacenter traffic to reduce the transmission cost," *IEEE/ACM Transactions on Networking*, vol. 26, no. 3, pp. 1193–1206, 2018.

[25] Z. Zhou, L. Pan, and S. Liu, "Online traffic allocation for video service providers in cloud-edge cooperative systems," *IEEE Transactions on Services Computing*, pp. 1–10, 2025.

[26] L. Luo, H. Yu, K.-T. Foerster, M. Noormohammadpour, and S. Schmid, "Inter-datacenter bulk transfers: Trends and challenges," *IEEE Network*, vol. 34, no. 5, pp. 240–246, 2020.

[27] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven wan," in *Proceedings of the ACM SIGCOMM*, 2013, pp. 15–26.

[28] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu *et al.*, "B4: Experience with a globally-deployed software defined wan," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 3–14, 2013.

[29] H. Liu, R. Urata, K. Yasumura, X. Zhou, R. Bannon, J. Berger, P. Dashti, N. Jouppi, C. Lam, S. Li *et al.*, "Lightwave fabrics: at-scale optical circuit switching for datacenter and machine learning systems," in *Proceedings of the ACM SIGCOMM*, 2023, pp. 499–515.

[30] H. Zhang, K. Chen, W. Bai, D. Han, C. Tian, H. Wang, H. Guan, and M. Zhang, "Guaranteeing deadlines for inter-data center transfers," *IEEE-ACM TRANSACTIONS ON NETWORKING*, vol. 25, no. 1, pp. 579–595, 2017.

[31] L. Luo, Y. Kong, M. Noormohammadpour, Z. Ye, G. Sun, H. Yu, and B. Li, "Deadline-aware fast one-to-many bulk transfers over inter-datacenter networks," *IEEE Transactions on Cloud Computing*, vol. 10, no. 1, pp. 304–321, 2019.

[32] L. Luo, K.-T. Foerster, S. Schmid, and H. Yu, "Dartree: Deadline-aware multicast transfers in reconfigurable wide-area networks," in *IEEE/ACM IWQoS*, 2019, pp. 1–10.

[33] M. Noormohammadpour, C. S. Raghavendra, S. Kandula, and S. Rao, "Quickcast: Fast and efficient inter-datacenter transfers using forwarding tree cohorts," in *IEEE INFOCOM 2018*, pp. 225–233.

[34] X. Lyu, H. Tian, W. Ni, Y. Zhang, P. Zhang, and R. P. Liu, "Energy-efficient admission of delay-sensitive tasks for mobile edge computing," *IEEE Transactions on Communications*, vol. 66, no. 6, pp. 2603–2616, 2018.

[35] S. Kim and P. K. Varshney, "An integrated adaptive bandwidth-management framework for qos-sensitive multimedia cellular networks," *IEEE Transactions on Vehicular technology*, vol. 53, no. 3, pp. 835–846, 2004.

[36] S. Lenhart and D. Fox, "Participatory democracy in dynamic contexts: A review of regional transmission organization governance in the united states," *Energy Research & Social Science*, vol. 83, p. 102345, 2022.

[37] Y. Dinitz, N. Garg, and M. X. Goemans, "On the single-source unsplittable flow problem," *Combinatorica*, vol. 19, no. 1, pp. 17–41, 1999.

[38] B. Ghojogh, A. Ghodsi, F. Karray, and M. Crowley, "Kkt conditions, first-order and second-order optimization, and distributed optimization: tutorial and survey," *arXiv preprint arXiv:2110.01858*, 2021.

[39] A. Mitsos, B. Chachuat, and P. I. Barton, "Mccormick-based relaxations of algorithms," *SIAM Journal on Optimization*, vol. 20, no. 2, pp. 573–601, 2009.

[40] H. Xu, Z. Yu, X.-Y. Li, C. Qian, L. Huang, and T. Jung, "Real-time update with joint optimization of route selection and update scheduling for sdns," in *2016 ICNP*. IEEE, 2016, pp. 1–10.

[41] C. Li, Q. Cai, and Y. Lou, "Optimal data placement strategy considering capacity limitation and load balancing in geographically distributed cloud," *Future Generation Computer Systems*, vol. 127, pp. 142–159, 2022.

[42] "Huawei cloud products," https://www.huaweicloud.com/product/ecs.html, MAY 2025.

[43] A. cluster data, https://github.com/alibaba/clusterdata/, MAY 2025.

APPENDIX A
NOTATION SEMANTICS

We summarize and list the important notations in Table II.

TABLE II: Important Notations

Notations	Semantics
Regional Information	
$G = (V, E)$	the original formulated graph
$G' = (V', E')$	the augmented graph
$R = \{r\}$	the set of cloud regions
C_r	the resource capacity of region r, which is measured by the number of CPU cores in this paper
p_r	the unit electricity price of region r
$p_{u,w}$	the unit bandwidth cost for a link (u, w)
$\tau_{u,w}$	the corresponding delay for a link (u, w)
$b_{u,w}$	the bandwidth capacity for a link (u, w)
E_1	the total electricity costs of all tasks
E_2	the total bandwidth costs of all tasks
Tenant Requirements	
$T = \{t\}$	the set of cloud tenants
$I = \{i\}$	the set of tasks
I^t	the set of tasks of tenant t
c_i^t	the resource required by task i of tenant t, which is measured by the number of CPU cores in this paper
q_i^t	the electricity consumption of task i
τ_r^t	the access delay between tenant t and region r
τ_i^t	the access delay demand between tenant t and task i
J	the communication demand set
$j = (i, i')$	the communication demand denoting that task i needs to communicate with i'
s_j	the source node of j
d_j	the sink node of j
b_j	the bandwidth demand of j
τ_j	the delay demand of j
Variables	
$x_{i,r}^t$	whether the task I_i^t is deployed in region r
$y_{u,w}^j$	whether the path of j is through link (u, w)
ζ	the lower-level optimization object, which is link load balancing ratio in this paper
Auxiliary Parameters of KKT conditions	
$\lambda_{u,w}$	the Lagrangian multiplier
α	the Lagrangian multiplier
β	the Lagrangian multiplier
$\Lambda_{u,w}$	variable $\lambda_{u,w} \sum_j y_{u,w}^j b_j$

APPENDIX B
DATASETS DESCRIPTIONS

We show the detailed information of evaluated datasets as follows.

- **Region Dataset**: We utilize a real-world datacenter deployment dataset provided by Huawei [42] to model the geographical distribution of regions. For scalability evaluation, we consider two configurations: a ***small-scale*** setting with 5 regions and a ***large-scale*** setting with 20 regions, while keeping other parameters consistent. The available inter-region bandwidth ranges from 1,000 to 10,000 Gbps, and the residual computing capacity in each region varies from 10,000 to 30,000 CPU cores. To approximate inter-region transmission latency, we collect the geographical coordinates of real-world cities representing each region and compute the Euclidean distances between them. These distances are subsequently mapped onto a normalized latency range of 5 ms to 100 ms. Electricity prices are determined according to regional geography; for example, regions located in the scarcity-resource zones are priced at approximately \$0.50/MWh, while regions in those zones with rich resources are priced at around \$0.38/MWh. The data transmission cost model follows Huawei Cloud's pricing scheme, with unit prices determined by the geographical locations of the source and destination regions. For instance, transmissions between cities within China mainland incur a cost of \$0.08/Gbps, while transmissions between China mainland and Hong Kong are charged at \$0.2642/Gbps.
- **Task Dataset**: We adopt the publicly available Alibaba Cluster Trace [43] as our task dataset, which has been widely used in prior research. The dataset includes information on user task requirements, including resource demands, inter-task communication volumes, and latency constraints. Each task demands between 30 and 50 CPU cores of computational resources. The transmission latency between interdependent tasks is strictly constrained within the range of 20 ms to 50 ms. Furthermore, the inter-task communication bandwidth requirement typically falls between 500 Mbps and 1,000 Mbps. To evaluate the effectiveness and scalability of the proposed algorithm under varying workload intensities, we consider two workload scales: the ***small-scale*** dataset includes 100 to 1,000 tenant task requests, and the ***large-scale*** dataset consists of 1,000 to 10,000 tenant task requests. These tasks are geographically distributed across the cities where regions are deployed and their neighboring areas. The transmission latency between users and regions is approximated by calculating the Euclidean distance between their respective locations, which is then mapped to a normalized interval of 5 ms to 100 ms.